How-To Library

Informatica

Use Case: Using Stateful Variables to Version Data in Informatica® Big Data Streaming

© Copyright Informatica LLC 2019, 2021. Informatica, the Informatica logo, and PowerExchange are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Abstract

You can use stateful variables to perform stateful computing in a streaming mapping. This article presents a use case about using stateful variables to version data. You can navigate the use case to understand how stateful variables can achieve stateful computing and to think about how you can employ stateful variables based on your unique requirements.

Supported Versions

- Big Data Streaming 10.2.2

Table of Contents

Overview. . . . 2
Use Case: Versioning Data. . . . 2
Solution Architecture. . . . 3
Change Data Capture using PowerExchange CDC Publisher. . . . 4
Versioning the Data using Stateful Variables. . . . 4
Read Operation: Reading Data from Kafka. . . . 5
Python Transformation: Applying a Pre-trained Model. . . . 5
Expression Transformation: Using Stateful Variables. . . . 6
Write Transformation: Preparing Mapping Outputs. . . . 8
Visualizing the Data with a Real-time Dashboard. . . . 9

Overview

Stateful variables are variables that store information about the state of the data. When an event arrives, the value that the variable stores is updated. If the events can be logically grouped, you can use partitions to store a separate state for each group.

For example, data on temperatures can be logically grouped by city, and you can use a stateful variable to store and update a city's average temperature as you stream new records.

You can also use stateful variables to retrieve the state of the data. You can use the state to define additional expressions or transformations that you want to apply to the data.

Use Case: Versioning Data

You work for a company that is developing a mobile application, or app, for automated same-day deliveries to customers. Customers can use the app to make a delivery, and the app creates a schedule and provisions a carrier from a company storage facility to the customer's doorstep.

When the app receives an update, a user's experience with the app can change. You want to collect user feedback about the updates and pinpoint when a user's sentiment changes to understand whether the update was well-received or poorly-received.

To collect user feedback, you prompt users for their name and the feedback string. The data is added to a relational table each time that a new record arrives.

The following table shows a few sample records that you might receive:

Name	Feedback
'Saul Steele'	'Best! App! EVER! Thank you so much for this great app. My life is so much better now'
'Angela Reyes'	'I learned about this app when I got a flyer in the mail so I checked it out. It's user friendly and there are so many things that I can get delivered. I never have to go to the store again'
'Damon Cunningham'	'My family and friends all stand by this app but every order is late. Last time the delivery finally arrived and I already left for vacation'

To simplify the data for analysis, you plan to transform the feedback into an identifier that indicates whether the user is satisfied or dissatisfied. The following table shows the transformation from user feedback to user sentiment:

Name	Sentiment
'Saul Steele'	'satisfied'
'Angela Reyes'	'satisfied'
'Damon Cunningham'	'dissatisfied'

Because a user can submit new feedback after each update, you also want to version the data for each user. To ensure that the data is versioned only when the same user sends a new record, you group the data by name.

When the same user sends feedback, the version of their data is incremented by 1. For example, the first time that Saul Steele submits feedback, the version of the data is 1. The second time that Saul submits feedback, the version of the data is 2.

To version the data, you can use stateful variables.

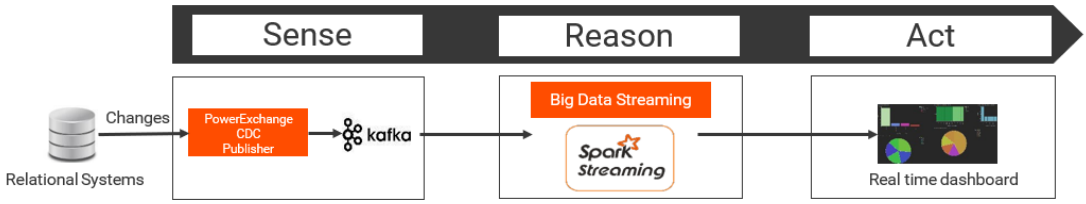
Solution Architecture

The solution architecture is built on the Sense-Reason-Act framework that is used to handle event-centric processing of streaming data. The framework uses the following steps:

1. Sense. An event is sensed, captured, and ingested.
2. Reason. Reasoning is applied using the dynamic context that is unique to each organization.
3. Act. The data is acted upon.

Dynamic context is built using a historical analysis of how data is used within the organization, such as the machine-learning (ML) models that the organization has developed or the set of business rules or patterns that the organization has established. Dynamic context is based on both the data that comes from within the organization, as well as data that comes from external sources, such as partners and social media.

Depending on the dynamic context for your organization, the following image shows one architecture that can solve this use case:



You can use the following components:

Sense: PowerExchange® CDC Publisher

Ingest the data using PowerExchange CDC Publisher to retrieve the change data and persist the change data in Kafka.

Reason: Big Data Streaming

Reason the data using Big Data Streaming to configure a stateful variable that captures the state of the change data as it streams from Kafka. The state can be used to assign versions to the incoming records and add value to the data.

Act: Real-time dashboard

Act on the data and use it in business decisions through a real-time dashboard that creates a comprehensive visual of the data. Different departments can use the visual to view an overall summary of the data for all users or more granular details for individual users.

Change Data Capture using PowerExchange CDC Publisher

Before you can use stateful variables to version the data, you must capture the change data and transfer it to Kafka so that you operate only on new records as they arrive.

To capture and transfer the change data, use PowerExchange CDC Publisher to retrieve the change data that PowerExchange captures from the relational table, format the change data into messages, and send the messages to Kafka.

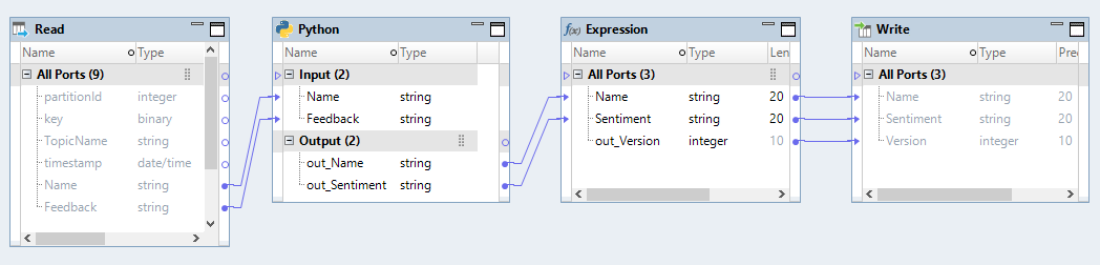
For more information about PowerExchange CDC Publisher, see the *Informatica PowerExchange CDC Publisher* documentation.

Versioning the Data using Stateful Variables

To use stateful variables to version the data in Big Data Streaming, create a streaming mapping and perform the following high-level steps:

1. Create a read operation using the Kafka data object to read the data from Kafka.
2. Optionally, use a Python transformation to apply a pre-trained sentiment-analysis model on the data.
3. Use an Expression transformation to version the data using stateful variables.
4. Use a Write transformation to write the data to the target.

The following image shows an overview of the streaming mapping:



Read Operation: Reading Data from Kafka

Using the Kafka data object, create a read operation and add the read operation to the streaming mapping. The read operation is used to stream the change data from Kafka and make the change data available to other transformations in the mapping.

The following input ports will be passed to other transformations in the mapping:

Input Port	Data Type
Name	String
Feedback	String

Python Transformation: Applying a Pre-trained Model

Optionally, you can use a Python transformation to operationalize a pre-trained sentiment-analysis model on the streaming data as the data moves through the mapping pipeline. The model transforms a user's feedback into an indicator of their sentiment. For each feedback string, the sentiment is either "satisfied" or "dissatisfied."

To apply the pre-trained model, add a Python transformation.

In the Python transformation, perform the following tasks:

1. Use the `Name` and `Feedback` ports as input ports, and create two output ports: `out_Name` and `out_Sentiment`.
2. Apply the pre-trained model to the input ports and store the results of the model in the output ports.

For example, the following table shows the first 10 input records that might have been sampled:

Name	Feedback
'Saul Steele'	'Best! App! EVER! Thank you so much for this great app. I've been using it since day one. My life is so much better now'
'Angela Reyes'	'I learned about this app when I got a flyer in the mail so I checked it out. It's user friendly and there are so many things that I can get delivered. I never have to go to the store again'
'Damon Cunningham'	'My family and friends all stand by this app but every order is late. Last time the delivery finally arrived and I already left for vacation'
'Saul Steele'	'The competition can't even compare to how easy it is to use this app! WOW'

Name	Feedback
'Angela Reyes'	'Admirable. hassle free'
'Erick Castillo'	'why can't I pick a delivery time'
'Constance Burke'	'It's very convenient. I can order things I need on the same day. I didn't know it was going to rain today and forgot to buy an umbrella but here it is!'
'Damon Cunningham'	'Finally I got an order that wasn't late!'
'Constance Burke'	'Super nice. I get a lot of coupons for everything I need.'
'Angela Reyes'	'I can't see my past orders anymore. The update must have changed it'

The following table shows the results of the model in the output records:

out_Name	out_Sentiment
'Saul Steele'	'satisfied'
'Angela Reyes'	'satisfied'
'Damon Cunningham'	'dissatisfied'
'Saul Steele'	'satisfied'
'Angela Reyes'	'satisfied'
'Erick Castillo'	'dissatisfied'
'Constance Burke'	'satisfied'
'Damon Cunningham'	'satisfied'
'Constance Burke'	'satisfied'
'Angela Reyes'	'dissatisfied'

For more information about how to configure a Python transformation, see the *Informatica Developer Transformation Guide*.

Expression Transformation: Using Stateful Variables

To version the data using stateful variables, add an Expression transformation to the mapping.

In the Expression transformation, perform the following steps:

1. Define the input ports to determine the data that you want to version.
2. Partition the records into groups based on a user's name.
3. Create a stateful variable port that tracks the version of each user's records.
4. Define the output ports to persist the version number for each record.

Step 1. Define Inputs for Versioning

Determine the data that you want to version by creating the following input ports in the Expression transformation:

Input Port	Data Type
`Name`	`String`
`Sentiment`	`String`

Then, link the output ports `out_Name` and `out_Sentiment` in the Python transformation to the input ports `Name` and `Sentiment` in the Expression transformation.

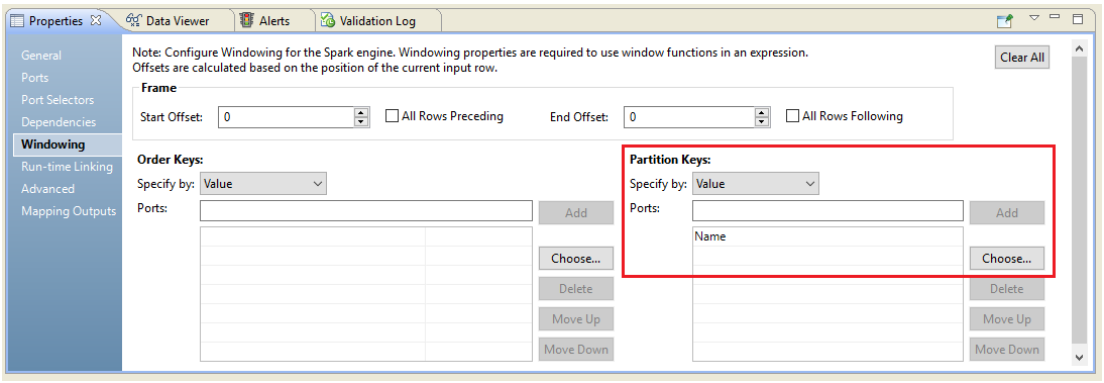
Step 2. Partition the Data

Create partitions to group the data based on a user's name.

The partitions are used to track a unique value in the stateful variable for each user. For example, the partition that contains Saul Steele's records will track a version number that is separate from the partition that contains Angela Reyes's records.

In the Expression transformation, use the **Windowing** tab to partition the data using the `Name` port. To configure the partitions, specify the `Name` port as the partition key.

The following image shows the configuration on the **Windowing** tab:

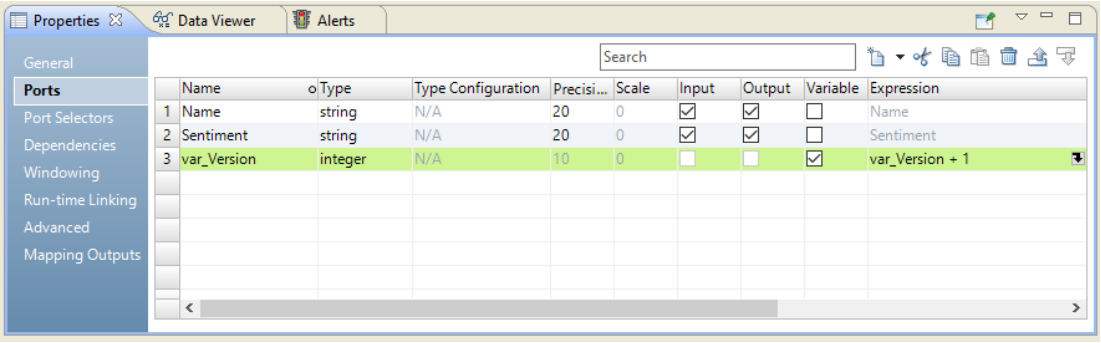


Step 3. Create a Stateful Variable Port

Configure the stateful variable port to track the version of each user's records.

On the **Ports** tab, create the variable port `var_Version` and configure the port expression `var_Version + 1`.

The following image shows the variable port `var_Version` and the port expression:



The initial state value of the port is 0. The first time that a record arrives from a specific user, the value in the port is 1. The second time that a record arrives from the same user, the value in the port becomes 2.

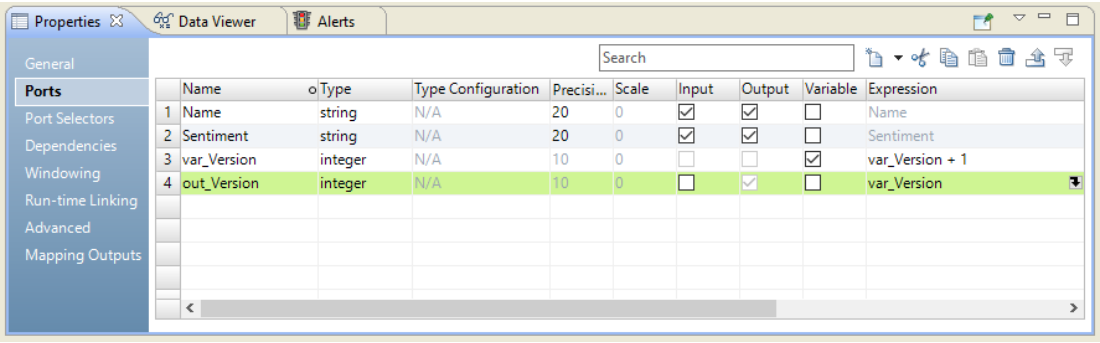
Step 4. Define Outputs

Define output ports to pass the versioned records to subsequent transformations. The versioned records include the user's name, sentiment, and record version number. In this scenario, the output ports will be passed to a Write transformation.

To pass a user's name and sentiment, make sure that the `Name` and `Sentiment` ports are marked as output ports.

To pass the version number, create an output port `out_Version` and configure the port expression `var_Version`.

The following image shows the output ports:



When data passes through the Expression transformation, the output port `out_Version` persists the latest value in the stateful variable port `var_Version`. For example, if the value of the `var_Version` port has been incremented three times, the value in the `out_Version` port is 3.

Write Transformation: Preparing Mapping Outputs

After you configure stateful variables to version the data, you can prepare the mapping to write the versioned output records to a target database.

To write the versioned output to a target, create a Write transformation that writes data to the target. Link the output ports in the Expression transformation to the Write transformation.

The Write transformation contains the following ports:

Port	Data Type
Name	String
Feedback	String
Version	Integer

The following table shows the mapping output based on the first 10 sample records:

Name	Sentiment	Version
‘Saul Steele’	'satisfied'	1
‘Angela Reyes’	'satisfied'	1
‘Damon Cunningham’	'dissatisfied'	1
‘Saul Steele’	'satisfied'	2
‘Angela Reyes’	'satisfied'	2
‘Erick Castillo’	'dissatisfied'	1
‘Constance Burke’	'satisfied'	1
‘Damon Cunningham’	'satisfied'	2
‘Constance Burke’	'satisfied'	2
‘Angela Reyes’	'dissatisfied'	3

You can see that the user Angela Reyes sent three records. As each record arrives, the version number increases by 1. For example, Angela's third record has a version number of 3.

When the data becomes available in the target, you can use the data in a real-time dashboard

Visualizing the Data with a Real-time Dashboard

You can display the data in a real-time dashboard that different departments in your company can access. For example, development might want to examine customers that used to be satisfied and became dissatisfied due to a recent update. Similarly, customer support might want to cater services to individuals that have been dissatisfied after multiple updates in a row.

Author

Margarita Pelyushenko